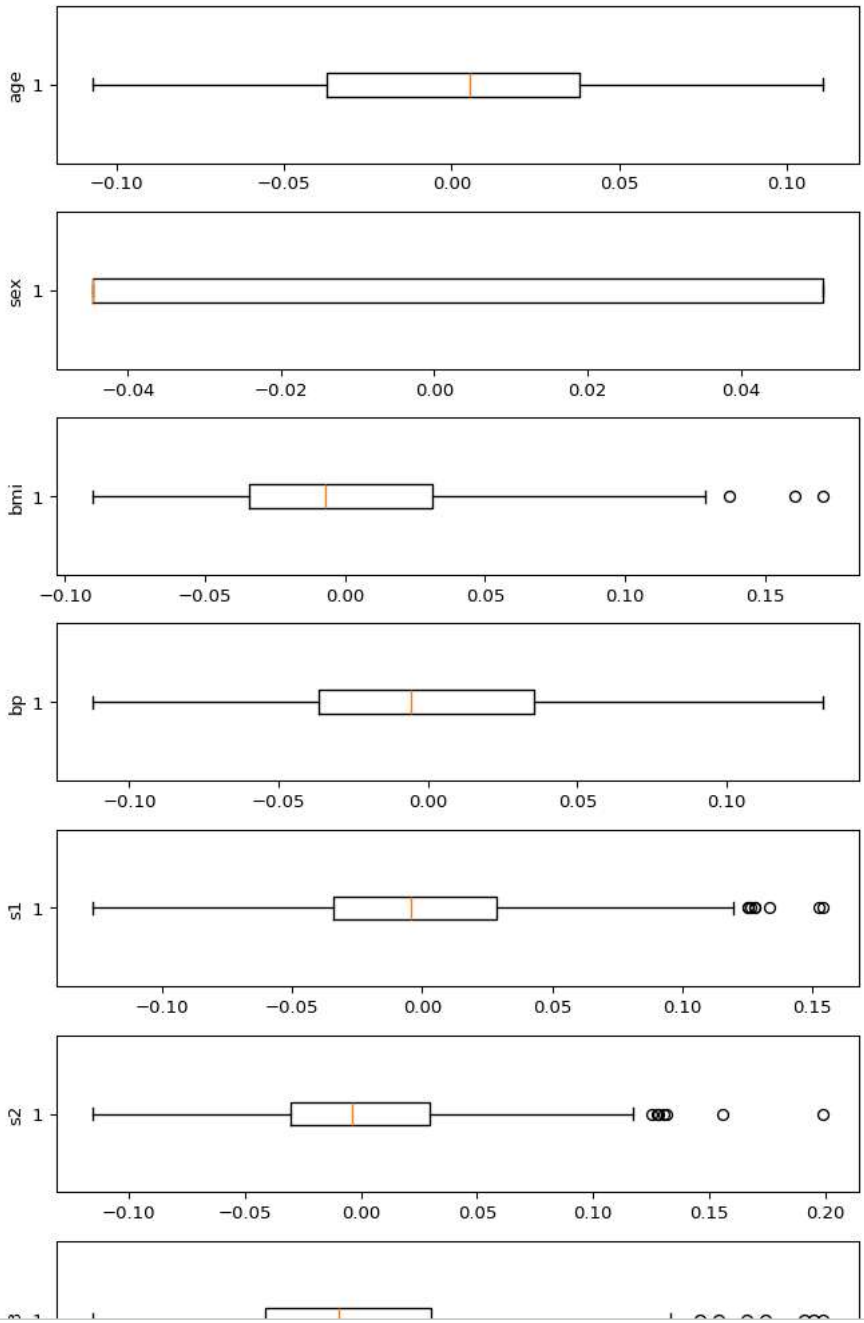
```
#Step-1
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler, StandardScaler
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.datasets import load_diabetes
diabetes = load_diabetes()
df = pd.DataFrame(diabetes.data, columns=diabetes.feature_names)
df['target'] = diabetes.target
df.head()
```

	age	sex	bmi	bp	s1	s2	s3	s4	s5	s6	target
0	0.038076	0.050680	0.061696	0.021872	-0.044223	-0.034821	-0.043401	-0.002592	0.019907	-0.017646	151.0
1	-0.001882	-0.044642	-0.051474	-0.026328	-0.008449	-0.019163	0.074412	-0.039493	-0.068332	-0.092204	75.0
2	0.085299	0.050680	0.044451	-0.005670	-0.045599	-0.034194	-0.032356	-0.002592	0.002861	-0.025930	141.0
3	-0.089063	-0.044642	-0.011595	-0.036656	0.012191	0.024991	-0.036038	0.034309	0.022688	-0.009362	206.0
4	0.005383	-0.044642	-0.036385	0.021872	0.003935	0.015596	0.008142	-0.002592	-0.031988	-0.046641	135.0

```
#Step-2
df.info()
print(df.isnull().sum())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 442 entries, 0 to 441
Data columns (total 11 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   age     442 non-null    float64
 1   sex     442 non-null    float64
 2   bmi     442 non-null    float64
 3   bp      442 non-null    float64
 4   s1      442 non-null    float64
 5   s2      442 non-null    float64
 6   s3      442 non-null    float64
 7   s4      442 non-null    float64
 8   s5      442 non-null    float64
 9   s6      442 non-null    float64
 10  target  442 non-null    float64
dtypes: float64(11)
memory usage: 38.1 KB
age       0
sex       0
bmi       0
bp        0
s1        0
s2        0
s3        0
s4        0
s5        0
s6        0
target    0
dtype: int64
```

```
#Step-3
fig, axs = plt.subplots(len(df.columns), 1, figsize=(7, 18), dpi=95)
for i, col in enumerate(df.columns):
  axs[i].boxplot(df[col], vert=False)
  axs[i].set_ylabel(col)
plt.tight_layout()
plt.show()
```

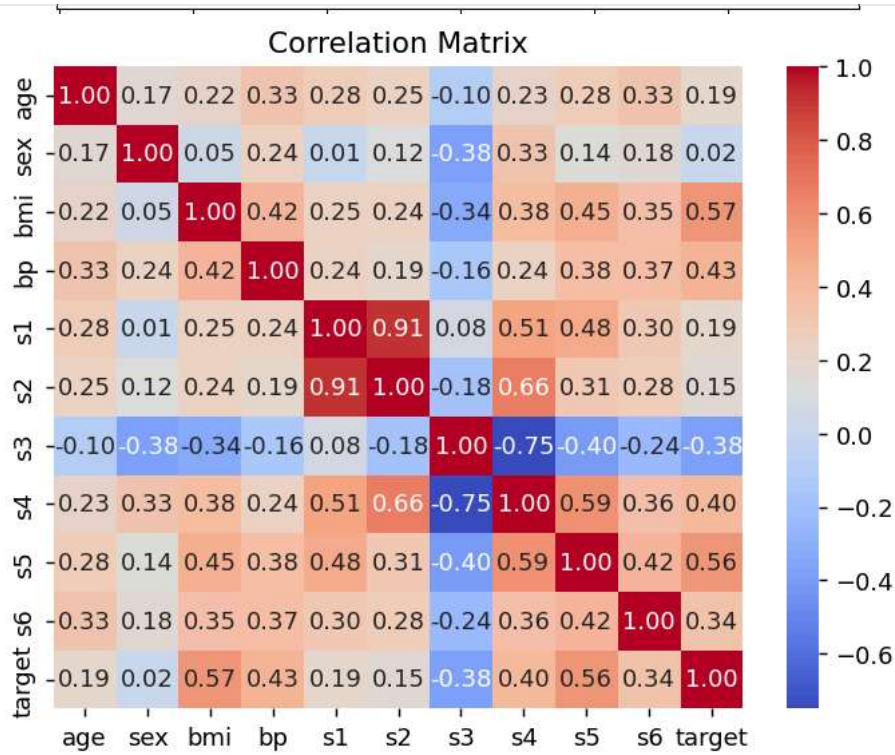



```
#Step-4:
numeric_cols = df.drop(columns=['target']).columns

for col in numeric_cols:
    q1 = df[col].quantile(0.25)
    q3 = df[col].quantile(0.75)
    iqr = q3 - q1
    lower = q1 - 1.5 * iqr
    upper = q3 + 1.5 * iqr
    df = df[(df[col] >= lower) & (df[col] <= upper)]

print("Shape after outlier removal:", df.shape)
```

```
Shape after outlier removal: (407, 11)
```

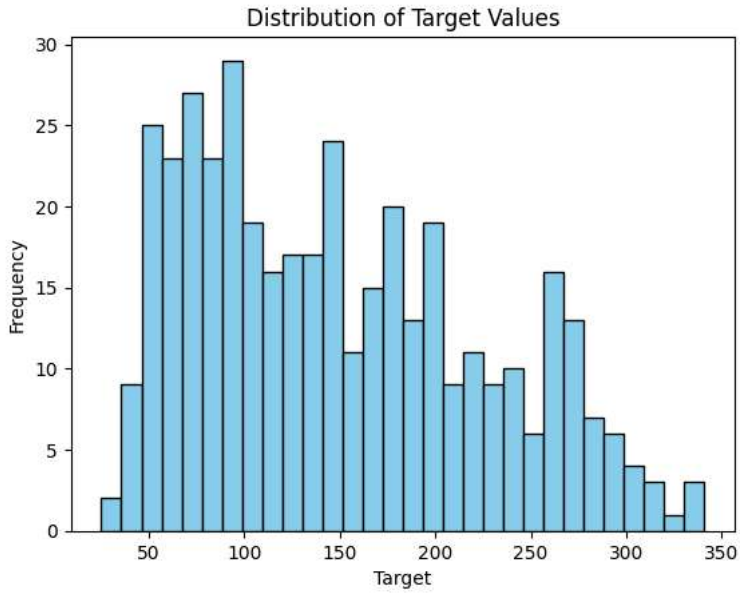
```
#Step-5
corr = df.corr()

plt.figure(dpi=130)
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm')
plt.title("Correlation Matrix")
plt.show()
```

```
print(corr['target'].sort_values(ascending=False))
```




```
target    1.000000
bmi       0.574610
s5        0.557329
bp        0.430632
s4        0.402010
s6        0.340406
age       0.193537
s1        0.192543
s2        0.153082
sex       0.022752
s3       -0.375491
Name: target, dtype: float64
```

```
#Step-6
plt.hist(df['target'], bins=30, color='skyblue', edgecolor='black')
plt.title('Distribution of Target Values')
plt.xlabel('Target')
plt.ylabel('Frequency')
plt.show()
```



```
#Step-7
X = df.drop(columns=['target'])
y = df['target']
```

```
#Step-8

# Normalization (Min-Max Scaling)
scaler = MinMaxScaler()
X_normalized = scaler.fit_transform(X)
print("Normalized sample:\n", X_normalized[:5])

# Standardization (Z-score Scaling)
scaler = StandardScaler()
X_standardized = scaler.fit_transform(X)
print("Standardized sample:\n", X_standardized[:5])
```

```
Normalized sample:
 [[0.66666667 1.         0.69458128 0.54929577 0.34883721 0.35983264
  0.26229508 0.33333333 0.59315632 0.38888889]
 [0.48333333 0.         0.1773399  0.35211268 0.5        0.42956764
  0.78688525 0.16666667 0.23467763 0.05555556]
 [0.88333333 1.         0.61576355 0.43661972 0.34302326 0.36262204
  0.31147541 0.33333333 0.52390475 0.35185185]
 [0.08333333 0.         0.35960591 0.30985915 0.5872093  0.62622036
  0.29508197 0.5        0.60445136 0.42592593]
 [0.51666667 0.         0.24630542 0.54929577 0.55232558 0.58437936
  0.49180328 0.33333333 0.38232789 0.25925926]]
Standardized sample:
 [[ 0.80565893  1.06343314  1.38124466  0.4831839  -0.90854505 -0.73667039
  -0.97747783  0.010936    0.52876202 -0.35329607]
 [-0.02677473 -0.9403506  -1.09453298 -0.55158235 -0.0838561  -0.37646417
   1.7453692  -0.83686474 -1.45028603 -2.03506798]
 [ 1.78944416  1.06343314  1.0039833  -0.1081111  -0.94026385 -0.72226214
  -0.72221092  0.010936    0.14644592 -0.54015961]
 [-1.84299361 -0.9403506  -0.2221161  -0.77331797  0.39192598  0.63931736
  -0.80729989  0.85873675  0.59111839 -0.16643252]
 [ 0.12457685 -0.9403506  -0.7644293   0.4831839   0.20161314  0.42319363
   0.21376774  0.010936   -0.63515539 -1.00731848]]
```